Python Introduction

Python is a **high-level, general-purpose** programming language with readable syntax.

1) What is Python ?

- **High-level**: code uses ideas such as lists and dictionaries; no manual memory management.
- **Interpreted workflow**: the Python runtime executes your code and reports errors.
- **Dynamically typed**: a name can refer to objects of different types at different times.
- **Multi-paradigm**: supports procedural, object-oriented and functional programming.

How Python runs code

Your code
(.py file or notebook cell)

↓

Python runtime
executes it line by line

↓

Output
or an error message

2) Why Python ?

- Readable syntax helps beginners focus on problem-solving.
- Large standard library and third-party ecosystem.
- Useful for automation, web backends, data analysis, AI/ML, testing and scripting.
- **Trade-off**: pure Python may be slower than compiled languages for CPU-intensive tasks.

3) Where is Python used ?

Area	Example
Automation	Rename files or generate reports
Web development	Build APIs and server-side applications
Data analysis	Clean and summarize datasets
AI / ML	Experiment with models
Testing	Automate checks for software
Education	Teach programming concepts

4) Run Python in VS Code

1. Install Python 3 and the Python extension
2. Select your interpreter
3. Save code in a .py file, then Run Python File in Terminal
4. In a notebook, run a cell with Shift + Enter

5) Your first program

Example :

```
print("Hello, Python!")
print("I am learning programming.")
print(2 + 3)
```

Output :

```
Hello, Python!
I am learning programming.
5
```

☆ REMEMBER

- Text in quotes is printed as written.
- `2 + 3` is calculated first, so it prints **5**.

Comments, Statements & Variables

Python groups code with **indentation** and stores values in **variables**.

1) Comments

- A comment begins with #.
- It explains the code and is not executed.

Example :

```
# This line is a comment
print("Python ignores the comment")
```

Output :

```
Python ignores the comment
```

2) Statements & Indentation

- Indentation groups statements into **blocks**.
- Use **four spaces** per indentation level.
- A colon : starts a block after if, for, def and class.

Example :

```
# This is a comment
student_name = "Asha"

if student_name:
    print(f"Hello, {student_name}!")
```

colon

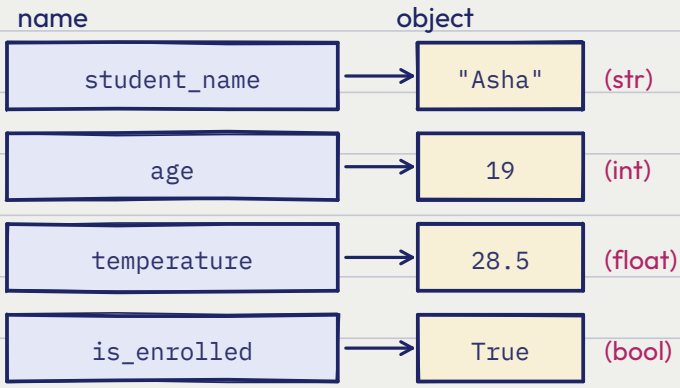
4 spaces

Output :

```
Hello, Asha!
```

3) Variables & Assignment

- A variable is a **name that refers to an object**.
- The = operator **assigns** a value to a name; it does not test equality.
- Names are **case-sensitive**: `score` and `Score` are different.
- Good names describe the purpose of the value.



Example :

```
student_name = "Asha"
age = 19
temperature = 28.5
is_enrolled = True

print(student_name)
print(age)
print(temperature)
print(is_enrolled)
```

Output :

```
Asha
19
28.5
True
```

Quick check

- Which symbol introduces an indented block? **Colon :**
- What does = do? **It assigns a value to a name; use `==` to test equality.**

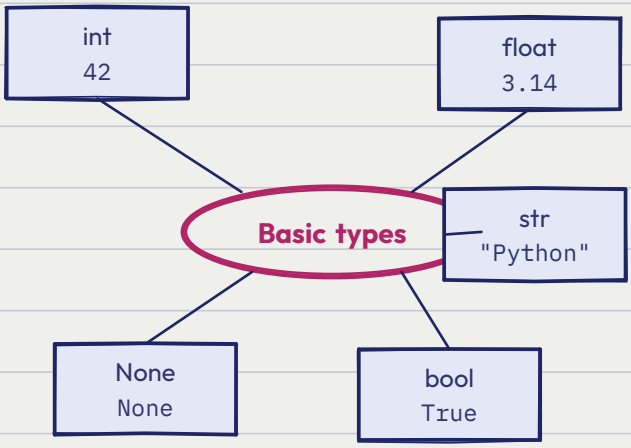
Try it : create variables for your name, favourite subject and weekly study hours, then print one sentence.

Built-in Data Types

Every value has a **type**. The type decides which operations make sense.

1) The five basic types

Type	Example	Typical use
`int`	`42`	Whole numbers
`float`	`3.14`	Decimal numbers
`str`	`"Python"`	Text
`bool`	`True`	Logical conditions
`NoneType`	`None`	No value / absence of a value



2) Check a type with type()

Example :

```
price = 49.50
language = "Python"
is_ready = False
not_set = None

print(type(price))
print(type(language))
print(type(is_ready))
print(type(not_set))
```

Output :

```
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

3) Dynamic typing

The same name can later refer to an object of a different type.

Example :

```
x = 42
print(type(x))
x = "Python"
print(type(x))
```

Output :

```
<class 'int'>
<class 'str'>
```

☆ REMEMBER

- `bool` has only two values: **True** and **False** (capital first letter).
- **None** means no value yet; its type is NoneType.

Quick check

What is `type(49.50)` ?

Answer: **float** (it has a decimal point)

Keywords & Identifiers

An **identifier** is a name you choose. A **keyword** is a word Python reserves for itself.

1) Identifier rules

- ✓ May contain letters, digits and underscores.
- ✓ Cannot begin with a digit.
- ✓ Cannot contain spaces or hyphens.
- ✓ Is case-sensitive.
- ✓ Cannot be a reserved keyword.

2) Valid vs invalid

Name	Valid ?	Why
`total_marks`	yes	letters + underscore
`_count`	yes	may start with _
`student2`	yes	digit is not first
`2score`	no	starts with a digit
`student-name`	no	hyphen not allowed
`class`	no	reserved keyword

3) Python keywords (cannot be used as names)

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Soft keywords such as **match** and **case** act as keywords only in specific syntax contexts.

Example :

```
# Valid identifiers
total_marks = 95
student2 = "Ravi"
_count = 3

print(total_marks, student2, _count)
```

Output :

```
95 Ravi 3
```

Using a keyword as a name :

```
class = "Python"
```

Python says :

```
SyntaxError: invalid syntax
```

Quick check

Which is a valid identifier: 2score, student-name, class, student2 ? **student2**

OPERATORS

Operators combine values. Examples below use **a = 7** and **b = 2**.

1) Arithmetic

Op	Meaning	Example	Result
+	addition	a + b	9
-	subtraction	a - b	5
*	multiplication	a * b	14
/	division	a / b	3.5
//	floor division	a // b	3
%	remainder	a % b	1
**	power	a ** b	49

2) Comparison

Op	Meaning	Example	Result
==	equal to	a == b	False
!=	not equal	a != b	True
<	less than	a < b	False
<=	less or equal	a <= b	False
>	greater than	a > b	True
>=	greater or equal	a >= b	True

3) Logical

Op	Meaning	Example
and	True only if both sides are True	a > b and b > 0 -> True
or	True if at least one side is True	a < b or b > 0 -> True
not	reverses True / False	not (a == b) -> True

☆ REMEMBER

- = assigns a value.
- == compares two values.

4) All together

Example :

```
a, b = 7, 2

print("Addition:", a + b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Remainder:", a % b)
print("Power:", a ** b)
print("Is a greater than b?", a > b)
print("Are they equal?", a == b)
```

Output :

```
Addition: 9
Division: 3.5
Floor division: 3
Remainder: 1
Power: 49
Is a greater than b? True
Are they equal? False
```

Predict first

`print(7 / 2, 7 // 2, 7 % 2)` Output: **3.5 3 1**

Quick check

What is `7 // 2` ? **3 (floor division rounds down; 7 / 2 = 3.5)**

Type Conversion, Input & Output

Convert values when an operation needs a different type. **input()** always gives text.

1) Type conversion

Call	What it does	Example	Result
`int()`	text / number to whole number	`int("19")`	19
`float()`	text / number to decimal	`float("1.72")`	1.72
`str()`	value to text	`str(19)`	"19"

☆ CAREFUL

- `int("hello")` raises **ValueError**.
- Validate user input.

Example :

```
age_text = "19"
age = int(age_text)
height = float("1.72")
message = "Age: " + str(age)

print(age + 1)
print(height)
print(message)
```

Output :

```
20
1.72
Age: 19
```

2) input() returns a string

User types 19 → input() gives "19" (str) → int() makes it 19 (int)

3) f-strings and safe input

An **f-string** starts with **f** and inserts values or expressions written inside **{ }**.

Example :

```
name = input("Your name: ").strip()
age_text = input("Your age: ").strip()

try:
    age = int(age_text)
except ValueError:
    print("Please enter your age as a whole number.")
else:
    print(f"Hello, {name or 'student'}!")
    print(f"Next year you will be {age + 1}.")
```

Run 1 (age = 19) :

```
Your name: Asha
Your age: 19
Hello, Asha!
Next year you will be 20.
```

Run 2 (age = abc) :

```
Your name: Asha
Your age: abc
Please enter your age as a whole number.
```

Quick check

`int("19") + 1` evaluates to ? **20**

What type does `input()` always return ? **str**

Practice Tasks & Solutions

Try each task in VS Code first. Then compare with the model solution.

Task 1

Variables for a book's title, price and availability.

Solution :

```
title = "Python Basics"
price = 349.50
in_stock = True
print(title, price, in_stock)
```

```
Python Basics 349.5 True
```

Task 2

Ask for two numbers and print their sum.

Solution :

```
a = float(input("First number: "))
b = float(input("Second number: "))
print("Sum:", a + b)
```

```
First number: 4
Second number: 5.5
Sum: 9.5
```

Task 3

Convert Celsius to Fahrenheit: F = C * 9 / 5 + 32.

Solution :

```
c = float(input("Celsius: "))
f = c * 9 / 5 + 32
print(f"{c} C = {f} F")
```

```
Celsius: 37
37.0 C = 98.6 F
```

Task 4

Tick the valid identifiers.

`first_name`	✓ valid
`first-name`	✗ hyphen
`2nd_place`	✗ starts with a digit
`class`	✗ keyword
`score2`	✓ valid

Task 5

Predict the output.

```
print(7 / 2, 7 // 2, 7 % 2)
```

Output: **3.5 3 1**

Common errors

Error	When it happens	Example
`SyntaxError`	a keyword is used as a name	`class = "Python"`
`ValueError`	text cannot be converted to a number	`int("hello")`

☆ QUICK RECAP

- Python is used for scripting, automation, web development, data and more.
- Variables are names bound to values / objects.
- Types determine what operations make sense.
- Keywords are reserved; identifiers are names you create.
- Input is text until converted.

Day 1 Cheat Sheet

Complete revision of Day 1 on one page. **Save and share!**

What Is Python

- High-level, general-purpose, readable
- Interpreted workflow
- Dynamically typed
- Multi-paradigm
- Automation, web, data, AI/ML, testing

Run Code In Vs Code

- Install Python 3 + extension
- Select interpreter
- Run Python File in Terminal
- Notebook: Shift + Enter

Syntax

- `#` starts a comment
- Indent with four spaces
- Colon `:` opens a block
- Names are case-sensitive

Variables

- `name = value` (assignment)
- `=` assigns, `==` compares
- `student_name = "Asha"`
- Name refers to an object

Data Types

- `int` 42 `float` 3.14
- `str` "Python" `bool` True
- `None` (NoneType)
- `type(x)` shows the type

Identifiers

- Letters, digits, _
- Cannot start with a digit
- No spaces or hyphens
- Cannot be a keyword

Operators

- `+ - * / // % **`
- `== != < <= > >=`
- `and or not`
- `7 // 2 = 3 7 % 2 = 1`

Convert & Input

- `int() float() str()`
- `input()` returns str
- `int("19") + 1` -> 20
- `int("hello")` -> ValueError

Output

- `print(a, b)`
- `f"Hello, {name}!"`
- f-string = readable output
- `print(2 + 3)` -> 5

Key Points To Remember

- ☆ Python is used for scripting, automation, web development, data and more.
- ☆ Variables are names bound to values / objects.
- ☆ Types determine what operations make sense.
- ☆ Keywords are reserved; identifiers are names you create.
- ☆ Input is text until converted.

Next lesson: Day 2 - Input, Formatting and Strings